

✓ Actividad Guiada 2 de Algoritmos de Optimización

Nombre: David Villacorta Nicolás

Cuaderno: https://colab.research.google.com/drive/1wSzuQoAt-hgamz2rSN-rLe_1wskPcDQ#scrollTo=ta1tvzVvsKPC

Proyecto GitHub: <https://github.com/davidvilla10/O3MIAR-Algoritmos-de-Optimizacion.git>

```
import math
import itertools
import time
import random

import matplotlib.pyplot as plt
import numpy as np

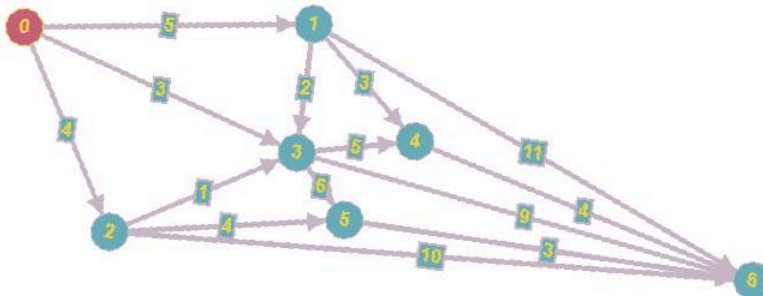
from sympy import symbols
from sympy.plotting import plot
from sympy.plotting import plot3d
from sympy import symbols, sin, cos, exp
```

✓ Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: “en una secuencia optima de decisiones, toda sub-secuencia también es óptima” (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.



*Consideramos una tabla $TARIFAS(i,j)$ para almacenar todos los precios que nos ofrecen los embarcaderos.

*Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
#Viaje por el río - Programación dinámica
#####

TARIFAS = [
[0,5,4,3,float("inf"),999,999], #desde nodo 0
[999,0,999,2,3,999,11], #desde nodo 1
[999,999, 0,1,999,4,10], #desde nodo 2
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#999 se puede sustituir por float("inf") del modulo math
TARIFAS

[[0, 5, 4, 3, inf, 999, 999],
[999, 0, 999, 2, 3, 999, 11],
[999, 999, 0, 1, 999, 4, 10],
```

```
[999, 999, 999, 0, 5, 6, 9],
[999, 999, 999, 999, 0, 999, 4],
[999, 999, 999, 999, 999, 0, 3],
[999, 999, 999, 999, 999, 999, 0]]
```

```
#Calculo de la matriz de PRECIOS y RUTAS
# PRECIOS - contiene la matriz del mejor precio para ir de un nodo a otro
# RUTAS - contiene los nodos intermedios para ir de un nodo a otro
#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in range(N)] #n x n
    RUTA = [ [""]*N for i in range(N)] # Irá guardando las rutas con los nodos intermedios

    #Se recorren todos los nodos con dos bucles(origen - destino)
    # para ir construyendo la matriz de PRECIOS
    for i in range(N-1):
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k
            PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
```

```
PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])
```

```
PRECIOS
[9999, 5, 4, 3, 8, 8, 11]
[9999, 9999, 999, 2, 3, 8, 7]
[9999, 9999, 9999, 1, 6, 4, 7]
[9999, 9999, 9999, 9999, 5, 6, 9]
[9999, 9999, 9999, 9999, 9999, 999, 4]
[9999, 9999, 9999, 9999, 9999, 9999, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

RUTA
['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']
```

```
#Calculo de la ruta usando la matriz RUTA
def calcular_ruta(RUTA, desde, hasta):
    if desde == RUTA[desde][hasta]:
        #if desde == hasta:
            #print("Ir a :" + str(desde))
            return desde
    else:
        return str(calcular_ruta(RUTA, desde, RUTA[desde][hasta])) + ', ' + str(RUTA[desde][hasta])

print("\nLa ruta pasa por :")
print(calcular_ruta(RUTA, 0,6) )
print("Con valor" , PRECIOS[0][6])
```

```
La ruta pasa por :
0,2,5
Con valor 11
```

▼ Problema de Asignacion de tarea

```
def calcular_tiempo(func):
    def wrapper(*args, **kwargs):
        inicio = time.time()
        resultado = func(*args, **kwargs)
        fin = time.time()
```

```

        print(f"Tiempo de ejecución: {fin - inicio:.6f} segundos")
    return resultado
return wrapper

```

```

#Asignacion de tareas - Ramificación y Poda
#####
#   T A R E A
#   A
#   G
#   E
#   N
#   T
#   E

COSTES=[[11,12,18,40],
        [14,15,13,22],
        [11,17,19,23],
        [17,14,20,28]]

```

```

#Calculo del valor de una solucion parcial
def valor(S,COSTES):
    VALOR = 0
    for i in range(len(S)):
        VALOR += COSTES[S[i]][i]
    return VALOR

```

```

valor((0, 1, 2, 3 ),COSTES)

```

73

```

#Coste inferior para soluciones parciales
# (1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al agente 1

def CI(S,COSTES):
    VALOR = 0
    #Valores establecidos
    for i in range(len(S)):
        VALOR += COSTES[i][S[i]]
    #Estimacion
    for i in range( len(S), len(COSTES) ):# recorremos los nodos que nos quedan por explorar
        VALOR += min( [ COSTES[j][i] for j in range(len(S), len(COSTES))  ])
    return VALOR

```

```

def CS(S,COSTES):
    VALOR = 0
    #Valores establecidos
    for i in range(len(S)):
        VALOR += COSTES[i][S[i]]

    #Estimacion
    for i in range( len(S), len(COSTES) ):# recorremos los nodos que nos quedan por explorar
        VALOR += max( [ COSTES[j][i] for j in range(len(S), len(COSTES))  ])
    return VALOR

```

```

CI((0,1),COSTES)

```

68

```

#Genera tantos hijos como como posibilidades haya para la siguiente elemento de la tupla
#(0,) -> (0,1), (0,2), (0,3)
def crear_hijos(NODO, N):
    HIJOS = []
    for i in range(N ):
        if i not in NODO:
            HIJOS.append({'s':NODO +(i,) })
    return HIJOS

```

```

crear_hijos((0,) , 4)

```

```

[{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]

```

```

@calcular_tiempo
def ramificacion_y_poda(COSTES):
    #Construccion iterativa de soluciones(arbol). En cada etapa asignamos un agente(ramas).
    #Nodos del grafo { s:(1,2),CI:3,CS:5 }
    #print(COSTES)
    DIMENSION = len(COSTES)
    MEJOR_SOLUCION=tuple( i for i in range(len(COSTES)) )
    CotaSup = valor(MEJOR_SOLUCION,COSTES)
    #print("Cota Superior:", CotaSup)

    NODOS=[]
    NODOS.append({'s':(), 'ci':CI((),COSTES) })

    iteracion = 0

```

```

while( len(NODOS) > 0):
    iteracion +=1

    nodo_prometedor = [ min(NODOS, key=lambda x:x['ci']) ][0]['s']
    #print("Nodo prometedor:", nodo_prometedor)

    #Ramificacion
    #Se generan los hijos
    HIJOS = [ {'s':x['s'], 'ci':CI(x['s'], COSTES)} for x in crear_hijos(nodo_prometedor, DIMENSION) ]

    #Revisamos la cota superior y nos quedamos con la mejor solucion si llegamos a una solucion final
    NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION ]
    if len(NODO_FINAL) >0:
        #print("\n*****Soluciones:", [x for x in HIJOS if len(x['s']) == DIMENSION ] )
        if NODO_FINAL[0]['ci'] < CotaSup:
            CotaSup = NODO_FINAL[0]['ci']
            MEJOR_SOLUCION = NODO_FINAL

    #Poda
    HIJOS = [x for x in HIJOS if x['ci'] < CotaSup ]

    #Añadimos los hijos
    NODOS.extend(HIJOS)

    #Eliminamos el nodo ramificado
    NODOS = [ x for x in NODOS if x['s'] != nodo_prometedor ]

print("La solucion final es:",MEJOR_SOLUCION , " en " , iteracion , " iteraciones" , " para dimension: " ,DIMENSION )

ramificacion_y_poda(COSTES)

```

La solucion final es: [{'s': (1, 2, 0, 3), 'ci': 64}] en 10 iteraciones para dimension: 4
 Tiempo de ejecución: 0.000313 segundos

¿A partir de qué dimensión el algoritmo por fuerza bruta deja de ser una opción?

Veremos cuál es el límite del algoritmo de fuerza bruta para resolver el problema de asignación de tareas. Para ello, lo pondremos a prueba generando matrices aleatorias de cada vez una dimensión mayor.

```

@calcular_tiempo
def fuerza_bruta(COSTES):
    # Representación de la solución será una tupla donde cada valor en la coordenada i-ésima es la tarea asignado al agente i
    # Ejemplo (1,2,3,4) tiene valor 11+15+19+28 = 73
    #
    # ¿Cuántas posibilidades hay? n! (porque al primer agente le podemos asignar n tareas, al segundo n-1, al tercero n-2, ...)

    mejor_valor = 10e10
    mejor_solucion = ()

    for s in list(itertools.permutations(range(len(COSTES)))):
        valor_tmp = valor(s,COSTES)
        if valor_tmp < mejor_valor:
            mejor_valor = valor_tmp
            mejor_solucion = s

    print("La mejor solución es :", mejor_solucion, "con valor: ", mejor_valor)

```

```

# Generamos una matriz NxN de números naturales aleatorios
def generar_costes(N, minimo=1, maximo=100):
    random.seed(42) # Para que el ejercicio sea reproducible
    COSTES = [[random.randint(minimo, maximo) for _ in range(N)] for _ in range(N)]
    return COSTES

for N in [5,6,7,8,9,10, 11]:
    print("\n=====")
    print(f"Probando N = {N}")
    COSTES = generar_costes(N)
    for fila in COSTES:
        print(fila)

    fuerza_bruta(COSTES)

```

Tiempo de ejecución: 0.116126 segundos

```
=====
Probando N = 9
[82, 15, 4, 95, 36, 32, 29, 18, 95]
[14, 87, 95, 70, 12, 76, 55, 5, 4]
[12, 28, 30, 65, 78, 4, 72, 26, 92]
[84, 90, 70, 54, 29, 58, 76, 36, 1]
[98, 21, 90, 55, 44, 36, 20, 28, 98]
[44, 14, 12, 49, 13, 46, 45, 78, 34]
[6, 94, 59, 69, 16, 49, 11, 71, 38]
[81, 80, 47, 74, 25, 91, 9, 6, 85]
[30, 99, 38, 11, 30, 13, 49, 36, 59]
La mejor solución es : (6, 4, 0, 8, 5, 2, 7, 1, 3) con valor: 74
Tiempo de ejecución: 1.075661 segundos

=====
Probando N = 10
[82, 15, 4, 95, 36, 32, 29, 18, 95, 14]
[87, 95, 70, 12, 76, 55, 5, 4, 12, 28]
[30, 65, 78, 4, 72, 26, 92, 84, 90, 70]
[54, 29, 58, 76, 36, 1, 98, 21, 90, 55]
[44, 36, 20, 28, 98, 44, 14, 12, 49, 13]
[46, 45, 78, 34, 6, 94, 59, 69, 16, 49]
[11, 71, 38, 81, 80, 47, 74, 25, 91, 9]
[6, 85, 30, 99, 38, 11, 30, 13, 49, 36]
[59, 82, 47, 21, 48, 46, 27, 86, 35, 90]
[88, 83, 10, 78, 82, 22, 69, 94, 32, 21]
La mejor solución es : (7, 0, 9, 2, 5, 3, 8, 4, 1, 6) con valor: 102
Tiempo de ejecución: 4.740765 segundos

=====
Probando N = 11
[82, 15, 4, 95, 36, 32, 29, 18, 95, 14, 87]
[95, 70, 12, 76, 55, 5, 4, 12, 28, 30, 65]
[78, 4, 72, 26, 92, 84, 90, 70, 54, 29, 58]
[76, 36, 1, 98, 21, 90, 55, 44, 36, 20, 28]
[98, 44, 14, 12, 49, 13, 46, 45, 78, 34, 6]
[94, 59, 69, 16, 49, 11, 71, 38, 81, 80, 47]
[74, 25, 91, 9, 6, 85, 30, 99, 38, 11, 30]
[13, 49, 36, 59, 82, 47, 21, 48, 46, 27, 86]
[35, 90, 88, 83, 10, 78, 82, 22, 69, 94, 32]
[21, 60, 49, 35, 82, 89, 72, 29, 88, 42, 99]
[100, 8, 30, 5, 41, 52, 35, 9, 28, 73, 92]
La mejor solución es : (7, 2, 3, 6, 8, 5, 1, 9, 10, 0, 4) con valor: 129
Tiempo de ejecución: 57.169696 segundos
```

El algoritmo por fuerza bruta puede ser razonable hasta $N = 10$. Cuando ejecutamos con $N = 11$, la ejecución ya dura 1 minuto aproximadamente. Y cuando ejecutamos con $N = 12$, nos salta el error por memoria RAM de Google Collab que se puede apreciar en la siguiente imagen:

Tu sesión ha fallado porque se ha usado toda la memoria RAM disponible.

[Ver registros del entorno de ejecución](#)



¿Hay algún valor de la dimensión a partir de la cual un algoritmo de ramificación y poda también deja de ser una opción válida?

Repetiremos con el algoritmo de ramificación y poda el proceso de generación de matrices de una dimensión mayor cada vez.

```
# Generamos una matriz NxN de números naturales aleatorios
def generar_costes(N, minimo=1, maximo=100):
    random.seed(42) # Para que el ejercicio sea reproducible
    COSTES = [[random.randint(minimo, maximo) for _ in range(N)] for _ in range(N)]
    return COSTES

for N in [5,6,7,8,9,10,11]:
    print("\n=====")
    print(f"Probando N = {N}")
    COSTES = generar_costes(N)
    for fila in COSTES:
        print(fila)

    ramificacion_y_poda(COSTES)
```

```

Probando N = 9
[82, 15, 4, 95, 36, 32, 29, 18, 95]
[14, 87, 95, 70, 12, 76, 55, 5, 4]
[12, 28, 30, 65, 78, 4, 72, 26, 92]
[84, 90, 70, 54, 29, 58, 76, 36, 1]
[98, 21, 90, 55, 44, 36, 20, 28, 98]
[44, 14, 12, 49, 13, 46, 45, 78, 34]
[6, 94, 59, 69, 16, 49, 11, 71, 38]
[81, 80, 47, 74, 25, 91, 9, 6, 85]
[30, 99, 38, 11, 30, 13, 49, 36, 59]
La solucion final es: [{'s': (2, 7, 5, 8, 1, 4, 0, 6, 3), 'ci': 74}] en 951 iteraciones para dimension: 9
Tiempo de ejecución: 0.142776 segundos

```

```

=====
Probando N = 10
[82, 15, 4, 95, 36, 32, 29, 18, 95, 14]
[87, 95, 70, 12, 76, 55, 5, 4, 12, 28]
[30, 65, 78, 4, 72, 26, 92, 84, 90, 70]
[54, 29, 58, 76, 36, 1, 98, 21, 90, 55]
[44, 36, 20, 28, 98, 44, 14, 12, 49, 13]
[46, 45, 78, 34, 6, 94, 59, 69, 16, 49]
[11, 71, 38, 81, 80, 47, 74, 25, 91, 9]
[6, 85, 30, 99, 38, 11, 30, 13, 49, 36]
[59, 82, 47, 21, 48, 46, 27, 86, 35, 90]
[88, 83, 10, 78, 82, 22, 69, 94, 32, 21]
La solucion final es: [{'s': (1, 6, 3, 5, 7, 4, 9, 0, 8, 2), 'ci': 103}] en 697 iteraciones para dimension: 10
Tiempo de ejecución: 0.085147 segundos

```

```

=====
Probando N = 11
[82, 15, 4, 95, 36, 32, 29, 18, 95, 14, 87]
[95, 70, 12, 76, 55, 5, 4, 12, 28, 30, 65]
[78, 4, 72, 26, 92, 84, 90, 70, 54, 29, 58]
[76, 36, 1, 98, 21, 90, 55, 44, 36, 20, 28]
[98, 44, 14, 12, 49, 13, 46, 45, 78, 34, 6]
[94, 59, 69, 16, 49, 11, 71, 38, 81, 80, 47]
[74, 25, 91, 9, 6, 85, 30, 99, 38, 11, 30]
[13, 49, 36, 59, 82, 47, 21, 48, 46, 27, 86]
[35, 90, 88, 83, 10, 78, 82, 22, 69, 94, 32]
[21, 60, 49, 35, 82, 89, 72, 29, 88, 42, 99]
[100, 8, 30, 5, 41, 52, 35, 9, 28, 73, 92]
La solucion final es: [{'s': (9, 6, 1, 2, 10, 5, 3, 0, 4, 7, 8), 'ci': 129}] en 22431 iteraciones para dimension: 11
Tiempo de ejecución: 79.547810 segundos

```

El algoritmo se ejecuta rápidamente para N = 5, 6, 7, 8, 9, y 10. Para N = 11 ronda el minuto. Para N = 12, a pesar de que no aparece el error de RAM que aparecía en el caso de Fuerza Bruta, el tiempo de ejecución se eleva a unos 9 minutos, lo que hace que este algoritmo deje de ser una opción válida.

```

La solucion final es: [{'s': (9, 10, 1, 0, 2, 8, 11, 6, 5, 7, 3, 4), 'ci': 115}] en 63722 iteraciones para dimension: 12
Tiempo de ejecución: 562.564463 segundos

```

Para N = 13, el tiempo de ejecución supera los 20 minutos:

 En ejecución (20 min 7 s)

Descenso del gradiente

Vamos a buscar el minimo de la funcion paraboloide por el método del descenso del gradiente :

$$f(x) = x^2 + y^2$$

Obviamente se encuentra en (x,y)=(0,0) pero probaremos como llegamos a él a través del descenso del gradiente.

```

#Definimos la funcion(f) y el gradiente(df)
#Paraboloide
f = lambda X:      X[0]**2 + X[1]**2      #Funcion
df = lambda X: [2*X[0] , 2*X[1]]          #Gradiente

```

```
df([1,2])
```

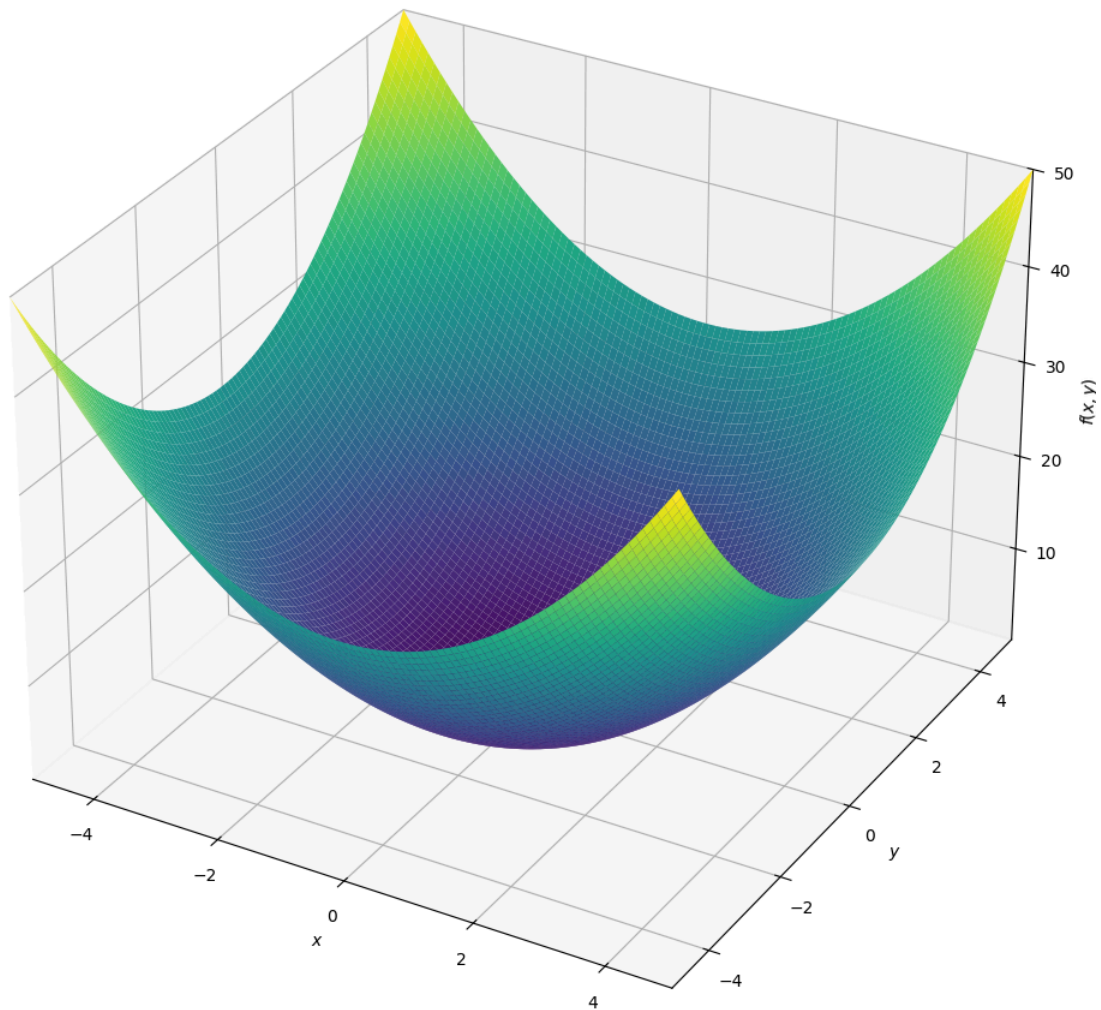
```
[2, 4]
```

```

x,y = symbols('x y')
plot3d(x**2 + y**2,
        (x,-5,5),(y,-5,5),
        title='x**2 + y**2',
        size=(10,10))

```

$$x^2 + y^2$$



<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x7f75cd7f5010>

```
#Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=2.5

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

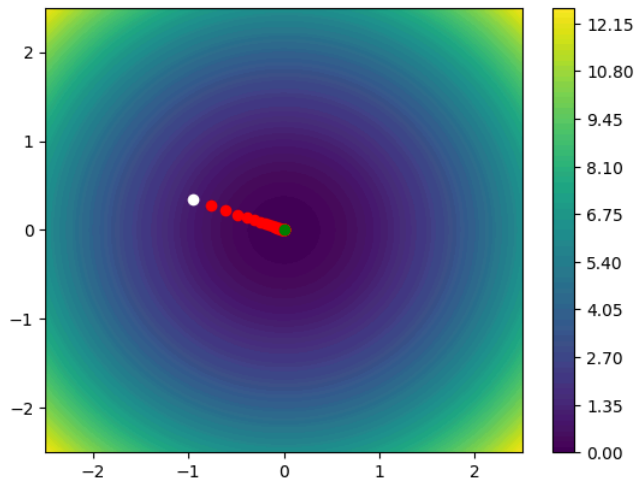
#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-2,2 ),random.uniform(-2,2 ) ] # Punto en el producto cartesiano [-2,2] x [-2,2]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acercamos.
TA=.1

#Iteraciones:500
for _ in range(50):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

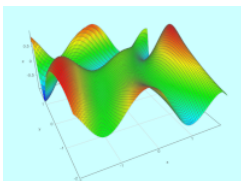
#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion: " , P , f(P)) # El punto P es de la forma [-1.35e-05, 4.82e-06], es decir, el algoritmo nos está descendiendo hasta el (0,
```



Solucion: [-1.3545059672430722e-05, 4.829006375271885e-06] 2.0678794410212555e-10

¿Te atreves a optimizar la función?:

$$f(x) = \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cos(2x + 1 - e^y)$$



Este ejercicio se podría hacer de manera aproximada con la definición de gradiente en un punto, o de manera analítica calculando la expresión del vector gradiente como hicimos en el caso anterior. Voy a optar por resolverlo de ambas formas para compararlas al final y confirmar que ambas son válidas.

✓ Opción A: Resolución analítica

- Derivada parcial respecto a x :

$$\frac{\partial f}{\partial x} = x \cos\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cos(2x + 1 - e^y) - 2 \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \sin(2x + 1 - e^y)$$

- Derivada parcial respecto a y :

$$\frac{\partial f}{\partial y} = -\frac{1}{2}y \cos\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cos(2x + 1 - e^y) + e^y \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \sin(2x + 1 - e^y)$$

```
#Definimos la funcion
f= lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3) *math.cos(2*X[0] + 1 - math.exp(X[1]) )

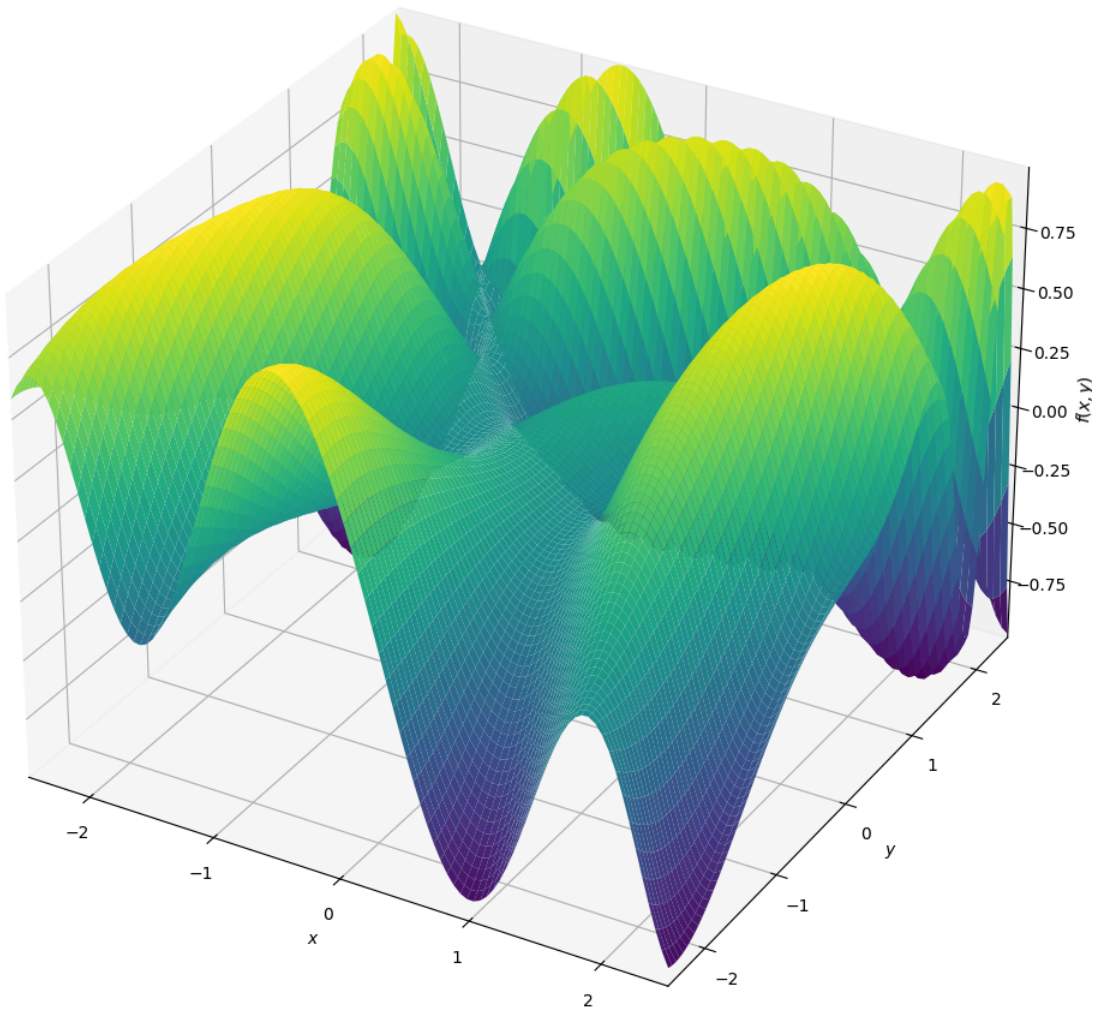
df = lambda X: [
    X[0] * math.cos(0.5 * X[0]**2 - 0.25 * X[1]**2 + 3) * math.cos(2 * X[0] + 1 - math.exp(X[1])) - 2 * math.sin(0.5 * X[0]**2 - 0.25 * X[1]**2 + 3) * math.sin(2 * X[0] + 1 - math.exp(X[1])) + math.exp(X[1]) * math.sin(0.5 * X[0]**2 - 0.25 * X[1]**2 + 3) * math.sin(2 * X[0] + 1 - math.exp(X[1]))
]
```

```
''' Visualizando la función vemos que no tiene un único mínimo (como sí le sucedía a la función f(x,y) = x^2 + y^2)'''
x, y = symbols('x y')

f_sym = sin(0.5*x**2 - 0.25*y**2 + 3) * cos(2*x + 1 - exp(y))

plot3d(
    f_sym,
    (x, -2.5, 2.5),
    (y, -2.5, 2.5),
    title='sin(0.5*x^2 - 0.25*y^2 + 3) * cos(2*x + 1 - exp(y))',
    size=(10,10)
)
```


$$\sin(0.5x^2 - 0.25y^2 + 3) * \cos(2x + 1 - \exp(y))$$



<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x7f771994b9b0>

```
#Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=4

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

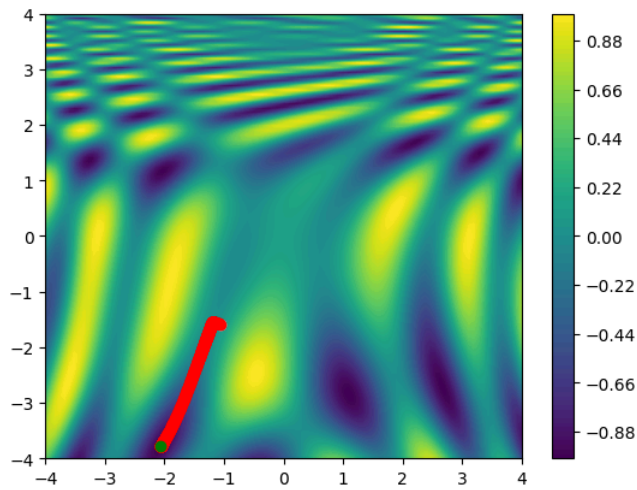
#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

random.seed(4) # Para que el ejercicio sea reproducible
#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-2,2 ),random.uniform(-2,2 ) ] # Punto en el producto cartesiano [-2,2] x [-2,2]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acercamos.
TA=0.02

for _ in range(1000):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))
```



Solucion: [-2.0592470898626085, -3.7679983117500147] -0.9999999999999654

```
P = [-2.0592470898626085, -3.7679983117500147]
print(df(P))
```

```
[1.493602244855702e-07, 2.6320925991459235e-07]
```

El algoritmo nos descende al punto $P = [-2.0592470898626085, -3.7679983117500147]$, que es un punto crítico porque, como vemos en la celda anterior, $\nabla f(P) = 0$.

Para concluir que es un mínimo voy a realizar un razonamiento usando conceptos que he visto en el Grado de Matemáticas. La función f es el producto de una función $\sin(t)$ por una función $\cos(t)$. Para cada $t \in \mathbb{R}$, se tiene que ambas funciones toman valores en el rango $[-1, 1]$. Por tanto, el mínimo valor que puede tomar f será -1 , y ocurrirá cuando el seno tome el valor 1 y el coseno -1 (o viceversa).

Como podemos ver en la imagen, el valor de $f(P)$ es 1 (salvo errores de redondeo):

Solucion: [-2.0592470898626085, -3.7679983117500147] -0.9999999999999654

Pero dado que f no puede tomar un valor menor a -1 por ser producto de funciones cuyo rango es $[-1, 1]$, entonces se verifica que $f(P) \leq f(x) \quad \forall x \in \mathbb{R}^2$. Precisamente esta es la definición de mínimo absoluto, y por tanto acabamos de demostrar que P también es un mínimo absoluto de la función. Probando con semillas diferentes para generar puntos de inicio diferentes, podemos encontrar otros puntos P' tales que $f(P') = -1$, es decir, más mínimos absolutos.

✓ Opción B: Resolución numérica

En lugar de utilizar la expresión analítica del gradiente de f , utilizamos la definición de derivada en un punto.

```
# Aproximamos la derivada de f con la definición de derivada en un punto (en lugar de calcular la derivada analítica)
def df_aproximado(PUNTO):
    h = 0.01
    T = np.copy(PUNTO)
    grad = np.zeros(2)

    for it in range(len(PUNTO)):
        T[it] = PUNTO[it] + h
        grad[it] = (f(T) - f(PUNTO)) / h
        T[it] = PUNTO[it] # Reseteamos este valor para no hacer el segundo cálculo desde un punto ya modificado.

    return grad
```

```
#Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=4

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

random.seed(4) # Para que el ejercicio sea reproducible
#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-2,2 ),random.uniform(-2,2 ) ] # Punto en el producto cartesiano [-2,2] x [-2,2]
plt.plot(P[0],P[1],"o",c="white")

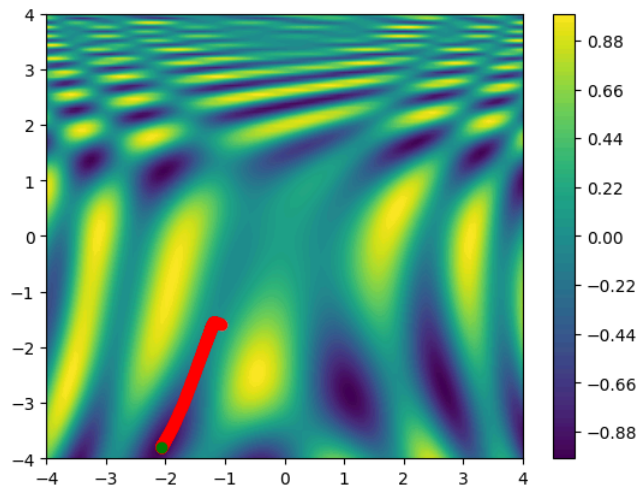
#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acercamos.
TA=0.02
```

```

for _ in range(1000):
    grad = df_aproximado(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))

```



Solucion: [np.float64(-2.0749482625696603), np.float64(-3.7903795031651524)] -0.9994745702540097

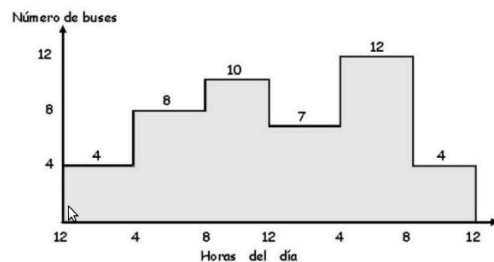
Empleando la misma semilla para partir del mismo punto inicial que en el método analítico, el método numérico de aproximar el vector gradiente de f en un punto, nos descende al mismo punto P que el método analítico:

Como vemos, el valor de la función f en este caso también se acerca a -1 , pero está algo más lejos que en el método analítico, debido a los errores de aproximación. Esto se debe a que no estamos usando exactamente la expresión del vector gradiente, sino una aproximación. Sin embargo, con esto concluimos que ambos métodos son correctos y nos conducen a un mínimo (local y absoluto) de la función f .

✓ Problema de Autobuses por Fuerza Bruta

Desarrollo e implementación de algoritmos. Ejemplo

Una pequeña ciudad estudia introducir un sistema de transporte urbano de autobuses. Nos encargan estudiar el número mínimo de autobuses para cubrir la demanda. Se ha realizado un estudio para estimar el número mínimo de autobuses por franja horaria. Lógicamente este número varía dependiendo de la hora del día. Se observa que es posible dividir la franja horaria de 24h en tramos de 4 horas en los que queda determinado el número de autobuses que se necesitan. Debido a la normativa cada autobús debe circular 8 horas como máximo y seguidas en cada jornada de 24h.



```

#####
# Ximo Sanz Tornero.
#####
from itertools import product

# Paso 1: Inicializamos los datos
# Demanda mínima de autobuses por tramo
demanda = [4, 8, 10, 7, 12, 4] # d[0], d[1], ..., d[5]
tramos = len(demanda) # Número de tramos (6 en este caso)

```

```

#Posible Solucion
x = [4,5,6,7,8,9]

for t in range(tramos):
    # Calculamos el número actual de autobuses que están cubriendo el tramo t
    cobertura_actual = x[t] + x[t - 1] # Autobuses en t y t-1 (cíclico)

    # Si la cobertura actual es menor que la demanda, añadimos autobuses en t
    if cobertura_actual < demanda[t]:
        # Añadimos los autobuses necesarios en el tramo t
        x[t] += demanda[t] - cobertura_actual

#Función objetivo
f_objetivo = sum(x)

# Se crean funciones tanto para verificar posibles soluciones como para obtener el valor de la función objetivo de la solución
# Función objetivo: suma de todos los x[i]
def funcion_objetivo(solucion):
    return sum(solucion)

# Verificación de restricciones
def verifica_restricciones(solucion):
    for t in range(tramos):
        # Se calcula el número actual de autobuses que están cubriendo el tramo t
        cobertura_actual = solucion[t] + solucion[t - 1] # Cíclico
        if cobertura_actual < demanda[t]:
            return False
    return True

# Se realiza el algoritmo por fuerza bruta con todas las posibles combinaciones de valores que tenemos en funcion de la variable demanda
# Generación del espacio de soluciones
# Para ver mejor el funcionamiento, ejecutar: for solucion in product(*(range(d + 1) for d in [1,2,3])): print(solucion)
espacio_soluciones = product(*(range(d + 1) for d in demanda))

# Búsqueda de la mejor solución
mejor_valor_objetivo = float('inf')
mejores_soluciones = []

for solucion in espacio_soluciones:
    if verifica_restricciones(solucion):
        valor_objetivo = funcion_objetivo(solucion)
        if valor_objetivo < mejor_valor_objetivo:
            mejor_valor_objetivo = valor_objetivo
            mejores_soluciones = [solucion]
        elif valor_objetivo == mejor_valor_objetivo:
            mejores_soluciones.append(solucion) # Agrega soluciones con el mismo valor

# Resultado
print("Mejores soluciones:")
for solucion in mejores_soluciones:
    print(solucion)
print("Valor mínimo de la función objetivo:", mejor_valor_objetivo)

```

Mejores soluciones:

```

(0, 8, 2, 5, 7, 4)
(0, 8, 2, 6, 6, 4)
(0, 8, 2, 7, 5, 4)
(1, 7, 3, 4, 8, 3)
(1, 7, 3, 5, 7, 3)
(1, 7, 3, 6, 6, 3)
(1, 7, 3, 7, 5, 3)
(1, 8, 2, 5, 7, 3)
(1, 8, 2, 6, 6, 3)
(1, 8, 2, 7, 5, 3)
(2, 6, 4, 3, 9, 2)
(2, 6, 4, 4, 8, 2)
(2, 6, 4, 5, 7, 2)
(2, 6, 4, 6, 6, 2)
(2, 6, 4, 7, 5, 2)
(2, 7, 3, 4, 8, 2)
(2, 7, 3, 5, 7, 2)
(2, 7, 3, 6, 6, 2)
(2, 7, 3, 7, 5, 2)
(2, 8, 2, 5, 7, 2)
(2, 8, 2, 6, 6, 2)
(2, 8, 2, 7, 5, 2)
(3, 5, 5, 2, 10, 1)
(3, 5, 5, 3, 9, 1)
(3, 5, 5, 4, 8, 1)
(3, 5, 5, 5, 7, 1)
(3, 5, 5, 6, 6, 1)
(3, 5, 5, 7, 5, 1)
(3, 6, 4, 3, 9, 1)
(3, 6, 4, 4, 8, 1)
(3, 6, 4, 5, 7, 1)
(3, 6, 4, 6, 6, 1)
(3, 6, 4, 7, 5, 1)
(3, 7, 3, 4, 8, 1)
(3, 7, 3, 5, 7, 1)
(3, 7, 3, 6, 6, 1)

```

```
(3, 7, 3, 7, 5, 1)
(3, 8, 2, 5, 7, 1)
(3, 8, 2, 6, 6, 1)
(3, 8, 2, 7, 5, 1)
(4, 4, 6, 1, 11, 0)
(4, 4, 6, 2, 10, 0)
(4, 4, 6, 3, 9, 0)
(4, 4, 6, 4, 8, 0)
(4, 4, 6, 5, 7, 0)
(4, 4, 6, 6, 6, 0)
(4, 4, 6, 7, 5, 0)
(4, 5, 5, 2, 10, 0)
(4, 5, 5, 3, 9, 0)
(4, 5, 5, 4, 8, 0)
(4, 5, 5, 5, 7, 0)
(4, 5, 5, 6, 6, 0)
(4, 5, 5, 7, 5, 0)
(4, 6, 4, 3, 9, 0)
(4, 6, 4, 4, 8, 0)
(4, 6, 4, 5, 7, 0)
(4, 6, 4, 6, 6, 0)
```

```
for solution in product(*(range(d + 1) for d in [1,2,3])): print(solution)
```

```
(0, 0, 0)
(0, 0, 1)
(0, 0, 2)
(0, 0, 3)
(0, 1, 0)
(0, 1, 1)
(0, 1, 2)
(0, 1, 3)
(0, 2, 0)
(0, 2, 1)
(0, 2, 2)
(0, 2, 3)
(1, 0, 0)
(1, 0, 1)
(1, 0, 2)
(1, 0, 3)
(1, 1, 0)
```